

LINUX: Servidor DHCP (Debian 12)

Servicios en Red e Internet · Versión 1 (septiembre de 2026)

LINUX: Servidor DHCP en Debian 12

Nota: todas las operaciones se hacen desde un terminal como administrador, preferiblemente anteponiendo sudo a cada orden.

Los nombres de interfaces y direcciones IP de los ejemplos pueden no coincidir con los de vuestro esquema de red. Adaptadlos siempre.

1. Configuración de las interfaces de red en Debian 12

Antes de montar cualquier servicio de red, el servidor tiene que tener su red bien configurada. En este apartado vemos cómo hacerlo **desde el terminal**, qué ficheros intervienen y cómo comprobar el resultado.

1.1. Identificar las interfaces

```
ip -br link          # interfaces y su estado (UP/DOWN) y MAC
ip -br address       # IP asignadas a cada interfaz
ip route             # tabla de rutas (la "default" es la puerta de
enlace)
```

Ejemplo de salida:

```
lo                UNKNOWN          127.0.0.1/8 ::1/128
ens18             UP                10.3.129.57/24
fe80::be24:11ff:fe4e:66a1/64
ens19             UP                192.168.29.1/24
fe80::be24:11ff:fe9b:1c2d/64
```

Los nombres de las interfaces dependen de dónde corre el sistema:

- **Máquina virtual o equipo físico:** Debian usa **nombres predecibles** según la posición de la tarjeta. Por ejemplo, ens18 es ethernet en la ranura (slot) 18; en las máquinas virtuales de Proxmox suelen ser ens18, ens19... y en VirtualBox enp0s3, enp0s8...
- **Contenedor LXC de Proxmox** (nuestro caso): el nombre **lo decide Proxmox**. Es el campo *Name* de cada tarjeta en la pestaña *Network* del contenedor. Por defecto propone eth0, eth1..., pero se puede poner cualquier otro. En nuestro escenario se llaman ens18 y ens19.

Sea cual sea el caso, comprobad siempre los nombres reales con `ip -br link` antes de configurar nada.

La orden `ifconfig` (paquete `net-tools`) está obsoleta y no viene instalada. Usamos siempre `ip`.

1.2. ¿Quién gestiona la red?

En Debian 12 hay dos formas habituales de gestionar la red, y **no deben gestionar la misma interfaz a la vez**:

	ifupdown	NetworkManager
Cuándo se usa	Instalación de servidor (sin escritorio)	Instalación con escritorio
Fichero principal	/etc/network/interfaces	/etc/NetworkManager/system-connections/*.nmconnection
Herramienta	ifup, ifdown	nmcli, nmtui
Servicio	networking	NetworkManager

NetworkManager **no toca** las interfaces que aparecen en `/etc/network/interfaces`. Para saber cuál tenéis:

```
systemctl is-active networking NetworkManager
```

En los servidores de clase usaremos **ifupdown**.

1.3. Configuración temporal con `ip`

Sirve para hacer pruebas rápidas. **Se pierde al reiniciar** el equipo o la red.

```
sudo ip link set ens19 up           # activar la
interfaz
sudo ip address add 192.168.29.1/24 dev ens19  # añadir una IP
sudo ip address del 192.168.29.1/24 dev ens19  # quitarla
sudo ip route add default via 10.3.129.1      # puerta de enlace
(la del router del aula)
sudo ip address flush dev ens19              # borrar todas las
IP de la interfaz
```

1.4. Configuración permanente: `/etc/network/interfaces`

Es el fichero principal de `ifupdown`. Ejemplo para nuestro servidor, con una interfaz por DHCP hacia Internet y otra estática hacia la red local:

```
# /etc/network/interfaces

source /etc/network/interfaces.d/*

# Loopback
```

```

auto lo
iface lo inet loopback

# Interfaz externa: recibe la configuración por DHCP
allow-hotplug ens18
iface ens18 inet dhcp

# Interfaz interna: IP estática en la red local
auto ens19
iface ens19 inet static
    address 192.168.29.1/24

```

Cuando la interfaz estática es la única del equipo, hay que añadir también la puerta de enlace y los DNS. Por ejemplo, el cliente estático del esquema (apartado 1.10), que sale a Internet a través del servidor:

```

auto ens18
iface ens18 inet static
    address 192.168.29.20/24
    gateway 192.168.29.1
    dns-nameservers 1.1.1.1 8.8.8.8
    dns-search sri.local

```

Significado de cada línea:

- **auto ens19:** activa la interfaz al arrancar el sistema.
- **allow-hotplug ens18:** la activa cuando el sistema detecta la tarjeta. Es lo que pone el instalador de Debian. En la práctica, para tarjetas fijas funciona igual que auto.
- **iface ens19 inet static:** configuración IPv4 (inet) estática. Con dhcp en lugar de static, la pide por DHCP. Para IPv6 se usa inet6.
- **address 192.168.29.1/24:** IP y máscara en notación CIDR. También vale la forma antigua con una línea netmask 255.255.255.0 aparte.
- **gateway:** puerta de enlace. **Solo debe haber una** en todo el fichero.
- **dns-nameservers** y **dns-search:** servidores DNS y dominio de búsqueda. **Solo funcionan si está instalado el paquete resolvconf.** Si no, se ignoran y hay que editar /etc/resolv.conf a mano (ver 1.6).
- **source /etc/network/interfaces.d/*:** incluye también los ficheros de esa carpeta, por si se prefiere un fichero por interfaz.

Aplicar los cambios:

```

sudo ifdown ens19 && sudo ifup ens19      # reinicia solo esa
interfaz
sudo systemctl restart networking         # reinicia todas

```

Cuidado: si estáis conectados por SSH, reiniciar la interfaz por la que entráis os puede dejar sin conexión si hay un error en el fichero.

Es mejor tocar la configuración desde la consola del equipo en Proxmox.

Si ifup responde que la interfaz ya está configurada, primero hay que hacer ifdown. Si se editó el fichero con la interfaz levantada y ifdown falla, se puede limpiar con `sudo ip address flush dev ens19` y después hacer ifup.

«ifup: orden no encontrada» al entrar como root con su: ifup, ifdown y otras órdenes de administración están en `/usr/sbin`, que no está en el PATH de un usuario normal. Si os hacéis root con su (sin guion), se conserva el PATH del usuario y la orden «no existe». Se nota porque el prompt sigue en vuestra carpeta: `root@debian:/home/usuario#`. Soluciones: entrar con **su** - (carga el entorno completo de root), usar **sudo** o escribir la ruta completa (`/usr/sbin/ifdown ens19`).

Comprobar qué paquete gestiona la red: `dpkg -l ifupdown ifupdown2`. En Debian 12, tanto en máquina virtual como en contenedor LXC, aparece instalado ifupdown (versión 0.8.x). ifupdown2 es la versión que usa el propio anfitrión Proxmox. Los dos leen `/etc/network/interfaces` y se manejan con ifup/ifdown.

Contenedores LXC de Proxmox (nuestro caso): en un contenedor LXC la red **la gestiona Proxmox**. Nombre, IP, puerta de enlace y DHCP de cada tarjeta se configuran desde la interfaz web (contenedor → *Network*). Proxmox **reescribe** `/etc/network/interfaces` al arrancar el contenedor, así que lo que editéis a mano se pierde al reiniciarlo. Tenéis dos opciones:

- Configurar la red desde Proxmox (lo recomendable): ens18 con IPv4 *DHCP* y ens19 con IPv4 *Static* 192.168.29.1/24, sin puerta de enlace. Después, abrid `/etc/network/interfaces` dentro del contenedor y veréis que Proxmox lo ha escrito igual que en el ejemplo de arriba.
- Editarlo a mano, creando antes el fichero vacío `/etc/network/.pve-ignore.interfaces` para que Proxmox no lo sobrescriba.

Los DNS (`/etc/resolv.conf`) y el nombre del equipo (`/etc/hostname`) también los gestiona Proxmox en los contenedores (pestaña *DNS* y opciones del contenedor).

1.5. Configuración con NetworkManager (nmcli)

Solo si el equipo usa NetworkManager, por ejemplo un cliente Debian con escritorio:

<code>nmcli device status</code>	# interfaces y
<code>conexión asociada</code>	
<code>nmcli connection show</code>	# conexiones
<code>definidas</code>	

```
# Pasar una conexión a IP estática
sudo nmcli connection modify "Wired connection 1" \
    ipv4.method manual \
    ipv4.addresses 192.168.29.20/24 \
    ipv4.gateway 192.168.29.1 \
    ipv4.dns "1.1.1.1 8.8.8.8"
sudo nmcli connection up "Wired connection 1"      # aplicar

# Volver a DHCP
sudo nmcli connection modify "Wired connection 1" \
    ipv4.method auto ipv4.addresses "" ipv4.gateway "" ipv4.dns ""
sudo nmcli connection up "Wired connection 1"
```

Si preferís un menú en modo texto, nmtui hace lo mismo.

1.6. DNS: /etc/resolv.conf

Indica a qué servidores DNS pregunta el equipo:

```
# /etc/resolv.conf
search sri.local
nameserver 1.1.1.1
nameserver 8.8.8.8
```

- Si una interfaz recibe su configuración por **DHCP**, este fichero **lo sobrescribe el cliente DHCP**, así que no merece la pena editarlo a mano.
- Si toda la configuración es estática y no está instalado resolvconf, se edita directamente.

1.7. Nombre del equipo: /etc/hostname y /etc/hosts

```
sudo hostnamectl set-hostname server2      # cambia /etc/hostname
hostnamectl                                 # comprobar
```

Después hay que actualizar la línea 127.0.1.1 de /etc/hosts, que es la que resuelve el nombre del propio equipo:

```
# /etc/hosts
127.0.0.1      localhost
127.0.1.1      server2.sri.local server2
```

1.8. El servidor como router de la red local

En nuestro escenario, los clientes usan el servidor (192.168.29.1) como puerta de enlace. Para que puedan salir a Internet a través de él hay que hacer dos cosas:

1. Activar el reenvío de paquetes IPv4, de forma permanente:

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-router.conf
sudo sysctl --system                # aplicar sin reiniciar
sysctl net.ipv4.ip_forward          # debe mostrar = 1
```

2. Hacer NAT (enmascaramiento) por la interfaz externa. Debian 12 usa **nftables** como cortafuegos por defecto (en lugar del antiguo iptables). Editamos /etc/nftables.conf:

```
#!/usr/sbin/nft -f

flush ruleset

table ip nat {
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;
        oifname "ens18" ip saddr 192.168.29.0/24 masquerade
    }
}

sudo apt install nftables                # si no está instalado
sudo nft -c -f /etc/nftables.conf        # comprobar la sintaxis
sudo systemctl enable --now nftables     # cargarlo ahora y en cada
arranque
sudo nft list ruleset                    # ver las reglas cargadas
```

1.9. Comprobaciones

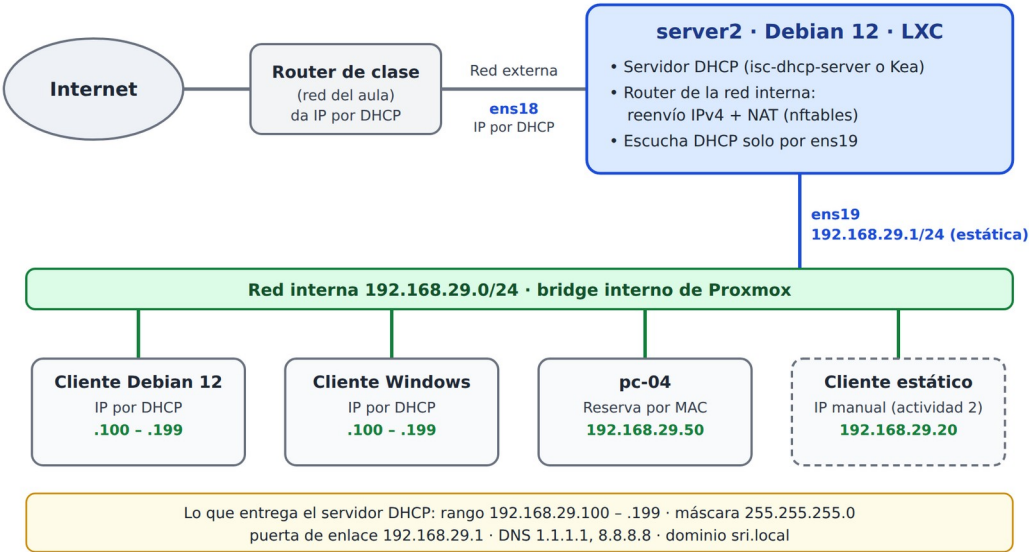
```
ip -br address                # ¿tiene cada interfaz la IP correcta?
ip route                      # ¿hay ruta por defecto?
ping -c 3 192.168.29.1        # conectividad en la red local
ping -c 3 1.1.1.1             # salida a Internet (sin DNS)
ping -c 3 deb.debian.org      # resolución DNS
getent hosts deb.debian.org    # qué IP resuelve
```

Si falla ping 1.1.1.1, el problema está en la **ruta o la puerta de enlace**. Si funciona ping 1.1.1.1 pero falla ping deb.debian.org, el problema es el **DNS**.

1.10. Esquema de red que vamos a montar

Esta es la red que montaremos en Proxmox. En las actividades 1 y 2 dejamos preparada la red del servidor y de un cliente; en los apartados siguientes montamos el servicio DHCP sobre ella.

Esquema de red del laboratorio



Esquema de red del laboratorio

Equipo	Interfaz	Red en Proxmox	Configuración
server2 (Debian 12)	ens18	Bridge externo (red del aula)	DHCP, recibida del router
server2 (Debian 12)	ens19	Bridge interno	Estática 192.168.29.1/24
Cliente Debian 12	ens18	Bridge interno	DHCP (rango .100-.199)
Cliente Windows	Ethernet	Bridge interno	DHCP (rango .100-.199)
pc-04	ens18	Bridge interno	DHCP con reserva 192.168.29.50
Cliente estático	ens18	Bridge interno	Manual 192.168.29.20/24, puerta de enlace 192.168.29.1

En Proxmox, la segunda tarjeta del servidor y la tarjeta de todos los clientes deben estar conectadas **al mismo bridge interno**; si no, no se verán entre ellos. El servidor (192.168.29.1) es su puerta de enlace, así que los clientes solo saldrán a Internet cuando el servidor tenga activados el reenvío y el NAT (apartado 1.8).

ACTIVIDAD 1: configura tu servidor con ens18 por DHCP y ens19 estática en 192.168.29.1/24 (si es un contenedor LXC, desde la

pestaña *Network* de Proxmox, ver 1.4), cambia su nombre a `server2` y comprueba la configuración con las órdenes de 1.9. Revisa qué ha quedado escrito en `/etc/network/interfaces`. Reinicia la máquina y verifica que todo se mantiene.

ACTIVIDAD 2: configura un cliente con IP estática `192.168.29.20/24` y puerta de enlace `192.168.29.1`. Activa el reenvío y el NAT en el servidor, y comprueba que el cliente sale a Internet. ¿Qué pasa si desactivas `net.ipv4.ip_forward`?

2. Antes de empezar: ¿qué software DHCP usamos?

El servidor DHCP «clásico» de Linux es **ISC DHCP** (paquete `isc-dhcp-server`), desarrollado por el Internet Systems Consortium. **ISC dejó de mantenerlo a finales de 2022** y recomienda su sucesor, **Kea** (paquetes `kea-dhcp4-server` y `kea-dhcp6-server`).

Debian 12 (Bookworm) todavía incluye `isc-dhcp-server` (versión 4.4.3), así que en estos apuntes lo usaremos para aprender los conceptos, porque su configuración es muy legible. En el **Anexo A** tenéis la misma configuración hecha con Kea, que es lo que encontraréis cada vez más en entornos reales.

	ISC DHCP	Kea
Paquete en Debian 12	<code>isc-dhcp-server</code>	<code>kea-dhcp4-server</code>
Estado	Sin mantenimiento desde 2022	Mantenido por ISC
Fichero de configuración	<code>/etc/dhcp/dhcpd.conf</code>	<code>/etc/kea/kea-dhcp4.conf</code> (JSON)
Concesiones	<code>/var/lib/dhcp/dhcpd.leases</code>	<code>/var/lib/kea/kea-leases4.csv</code>
Servicio systemd	<code>isc-dhcp-server</code>	<code>kea-dhcp4-server</code>

3. Escenario

Nuestro servidor, `server2`, tiene dos interfaces de red:

- **ens18:** dinámica, recibe su configuración del router que da salida a Internet.
- **ens19:** estática, conectada a la red local interna **192.168.29.0/24**, con la IP **192.168.29.1**.

El servicio DHCP escuchará **solo** por la interfaz interna, que **debe tener IP estática** antes de instalar nada. Configura la red del servidor tal como se explica en el apartado 1.4 y comprobala con las órdenes de 1.9.

4. Instalación

Actualizamos la lista de paquetes e instalamos el servidor:

```
sudo apt update
sudo apt install isc-dhcp-server
```

Al terminar, la instalación intenta arrancar el servicio y **falla**. Es normal: todavía no lo hemos configurado. Vemos por qué con:

```
sudo systemctl status isc-dhcp-server
sudo journalctl -u isc-dhcp-server -e
```

Cambio importante respecto a versiones antiguas: Debian 12 ya no instala **rsyslog** por defecto, así que **no existe** `/var/log/syslog`. Los registros se consultan con `journalctl`, que los filtra por servicio (`-u`), muestra el final (`-e`) o los sigue en directo (`-f`).

ACTIVIDAD 3: revisa con `journalctl -u isc-dhcp-server` los mensajes del servicio. ¿Qué errores aparecen? Coméntalo con tus compañeros.

En el registro veremos, principalmente, dos problemas:

- **No se ha indicado la interfaz** por la que el servicio debe escuchar, así que intenta arrancar en todas y además lanza a la vez los servidores de IPv4 e IPv6.
- **No hay ninguna declaración subnet** (ámbito) que coincida con las redes de las interfaces: No subnet declaration for ens19 (192.168.29.1).

4.1. Interfaz de escucha

Se indica en `/etc/default/isc-dhcp-server`. En versiones actuales hay **una variable para IPv4 y otra para IPv6** (la antigua `INTERFACES` ya no se usa):

```
INTERFACESv4="ens19"
INTERFACESv6=""
```

Si hay varias interfaces, se separan con espacios: `INTERFACESv4="ens19 ens20"`. Dejando `INTERFACESv6` vacía evitamos que intente arrancar también el servidor DHCPv6.

5. Configuración: `/etc/dhcp/dhcpd.conf`

El fichero que instala el paquete trae muchos ejemplos comentados que podéis usar como referencia. Conviene guardar una copia antes de editarlo:

```
sudo cp /etc/dhcp/dhcpd.conf /etc/dhcp/dhcpd.conf.orig
```

Ejemplo completo para nuestro escenario:

```
# /etc/dhcp/dhcpd.conf

# Este servidor es el DHCP oficial de la red: responde con DHCPNAK
# a clientes que pidan IPs que no son de esta red
authoritative;

# Parámetros globales (se aplican a todos los ámbitos)
default-lease-time 600;          # 10 minutos
max-lease-time 7200;             # 2 horas
option domain-name "sri.local";
option domain-name-servers 1.1.1.1, 8.8.8.8;

# Ámbito de la red local
subnet 192.168.29.0 netmask 255.255.255.0 {
    range 192.168.29.100 192.168.29.199;
    option routers 192.168.29.1;
    option broadcast-address 192.168.29.255;
}

# Reserva de IP para un equipo concreto
host pc-04 {
    hardware ethernet bc:24:11:4c:7f:1d;
    fixed-address 192.168.29.50;
}
```

Parámetros más importantes:

- **authoritative**: el servidor se considera el responsable de esa red. Sin él, un cliente que venga de otra red con una IP antigua tardará más en obtener una nueva.
- **default-lease-time**: duración de la concesión, en segundos, si el cliente no pide una concreta.
- **max-lease-time**: duración máxima que se concede aunque el cliente pida más.
- **option domain-name**: nombre de dominio que se entrega a los clientes.
- **option domain-name-servers**: servidores DNS, **separados por comas**.
- **subnet ... netmask ...**: declara un ámbito: la dirección de red y su máscara.
- **range**: intervalo de IP que se reparten dinámicamente. Tiene que estar **dentro de la red** del subnet. Puede haber varias líneas range.
- **option routers**: puerta de enlace (gateway) para los clientes.
- **option broadcast-address**: dirección de difusión (broadcast) de la red. Ojo: no es lo mismo que multidifusión (multicast).
- **option subnet-mask**: máscara que se entrega al cliente. Si no se pone, se usa la del subnet, así que normalmente no hace falta.
- **host**: define una **reserva**. El nombre es solo una etiqueta.

- **hardware ethernet:** MAC del equipo, escrita **con dos puntos** (bc:24:11:4c:7f:1d). El formato de Windows con guiones (BC-24-11-4C-7F-1D) da error.
- **fixed-address:** IP reservada. Se recomienda que esté **fuera** del range para no chocar con las dinámicas.
- **server-identifier** (opcional): IP con la que el servidor se identifica ante los clientes. Si el servidor tiene varias interfaces, se usa la de la interfaz por la que responde, así que casi nunca hace falta.

Errores frecuentes: comillas tipográficas (“sri.com”) copiadas desde un procesador de textos en lugar de comillas rectas ("sri.com"); olvidar un ;; poner un punto final en una IP (8.8.8.8.); rangos que se salen de la red del subnet.

5.1. Comprobar la sintaxis antes de reiniciar

Antes de reiniciar el servicio, comprobamos que el fichero es correcto:

```
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf
```

Si hay errores, indica la línea y marca con ^ dónde está el problema. Por ejemplo:

```
/etc/dhcp/dhcpd.conf line 3: semicolon expected.
option domain-name-servers 8.8.8.8.
                                ^
```

```
/etc/dhcp/dhcpd.conf line 8: expecting numeric value.
hardware ethernet 00-08-74-4C-7F-1D;
                                ^
```

Configuration file errors encountered -- exiting

ACTIVIDAD 4: introduce a propósito los dos errores anteriores en tu dhcpd.conf y comprueba qué te dice dhcpd -t. Después corrígelos.

6. Arrancar y parar el servicio

Los servicios se gestionan con **systemctl** (las órdenes antiguas /etc/init.d/... y service ... siguen funcionando por compatibilidad, pero ya no son la forma recomendada):

```
sudo systemctl start    isc-dhcp-server
sudo systemctl stop     isc-dhcp-server
sudo systemctl restart  isc-dhcp-server
sudo systemctl status   isc-dhcp-server
sudo systemctl enable   isc-dhcp-server # arrancar al iniciar el
sistema
```

Cada vez que cambiemos la configuración hay que reiniciar el servicio (no el servidor completo): `sudo systemctl restart isc-dhcp-server.`

Para ver en directo lo que hace el servidor mientras los clientes piden IP:

```
sudo journalctl -u isc-dhcp-server -f
```

Veremos el intercambio **DORA**: DHCPDISCOVER, DHCPOFFER, DHCPREQUEST y DHCPACK.

ACTIVIDAD 5: ejecuta `systemctl status isc-dhcp-server` y analiza la información que muestra: estado, PID, desde cuándo está activo y últimas líneas del registro.

7. Comprobación desde los clientes

7.1. Cliente Linux (Debian 12)

Con `ifupdown` y la interfaz configurada como `dhcp`:

```
sudo dhclient -r -v ens18    # libera la concesión actual
(DHCPRELEASE)
sudo dhclient -v ens18       # pide una nueva (se ve el DORA completo)
ip -br address               # IP recibida
ip route                     # puerta de enlace
cat /etc/resolv.conf         # DNS y dominio
```

Si el cliente usa `NetworkManager` (por ejemplo, Debian con escritorio):

```
nmcli device show ens18
sudo nmcli connection down "Wired connection 1" && sudo nmcli
connection up "Wired connection 1"
```

7.2. Cliente Windows 10/11

```
ipconfig /release
ipconfig /renew
ipconfig /all
```

ANALIZA: ejecuta `ipconfig /all` en un cliente Windows y localiza: IP, máscara, puerta de enlace, servidor DHCP, servidores DNS, sufijo DNS y fechas de concesión obtenida y de caducidad. ¿Cuadran con lo que configuraste en el servidor?

8. Información de concesiones (leases)

El servidor guarda las concesiones que va asignando en `/var/lib/dhcp/dhcpd.leases`:

```
lease 192.168.29.100 {
  starts 1 2026/09/28 15:34:51;
  ends 1 2026/09/28 15:44:51;
  cltt 1 2026/09/28 15:34:51;
  binding state active;
  next binding state free;
```

```

rewind binding state free;
hardware ethernet c6:4a:69:5f:32:e3;
client-hostname "cliente1";
}

```

Las fechas de este fichero están en **UTC**, así que en España veréis una o dos horas de diferencia con la hora local.

Para verlas en forma de tabla, el paquete incluye la herramienta `dhcp-lease-list`:

```
sudo dhcp-lease-list
```

ACTIVIDAD 6: libera la concesión de un cliente (`dhclient -r o ipconfig /release`) y comprueba qué cambia en `dhcpd.leases`. ¿Qué valor toma `binding state`?

Las **reservas** (host) no aparecen en este fichero, porque no son concesiones dinámicas.

9. Varias redes y pools

9.1. Un servidor con varias interfaces

Si el servidor está conectado a varias redes, basta con declarar **un subnet por red** y añadir todas las interfaces a `INTERFACESv4`:

```

# /etc/default/isc-dhcp-server
INTERFACESv4="ens19 ens20"

# Red de departamentos y despachos (ens19)
subnet 192.168.10.0 netmask 255.255.255.0 {
    range 192.168.10.50 192.168.10.100;
    option routers 192.168.10.1;
    option domain-name-servers 8.8.8.8, 1.1.1.1;
}

# Red de aulas (ens20)
subnet 192.168.20.0 netmask 255.255.255.0 {
    range 192.168.20.50 192.168.20.100;
    option routers 192.168.20.1;
    option domain-name-servers 8.8.8.8, 1.1.1.1;
}

```

El servidor sabe qué ámbito usar según la interfaz por la que le llega la petición.

9.2. Redes a las que el servidor no está conectado: agente relay

Las peticiones DHCP son **difusiones (broadcast)** y los routers no las **reenvían**. Para dar servicio a una red remota, el router de esa red debe

tener un **agente de retransmisión (DHCP relay)**, que reenvía las peticiones al servidor en unidifusión. En Linux lo proporciona el paquete `isc-dhcp-relay`; en routers Cisco es la orden `ip helper-address`. En el servidor solo hay que declarar el subnet de esa red remota.

9.3. shared-network

`shared-network` **no** sirve para agrupar redes distintas cualquiera. Se usa cuando **varias subredes IP comparten el mismo segmento físico** (por ejemplo, se ha quedado pequeña la red y se añade una segunda subred en el mismo cable o VLAN). Todas las subredes de dentro se tratan como una sola red:

```
shared-network aulas {
    subnet 192.168.20.0 netmask 255.255.255.0 {
        option routers 192.168.20.1;
    }
    subnet 192.168.21.0 netmask 255.255.255.0 {
        option routers 192.168.21.1;
    }
    pool {
        range 192.168.20.50 192.168.20.250;
    }
    pool {
        range 192.168.21.50 192.168.21.250;
    }
}
```

9.4. Pools: permitir o denegar clientes

Con `pool` se puede decidir **quién** recibe IP de cada rango. Un ejemplo útil: solo dar IP a los equipos que tienen reserva (`host`) y a nadie más:

```
subnet 192.168.29.0 netmask 255.255.255.0 {
    option routers 192.168.29.1;
    pool {
        deny unknown-clients;          # solo equipos declarados con
"host"
        range 192.168.29.100 192.168.29.199;
    }
}
```

ACTIVIDAD 7: configura el pool anterior y comprueba que un cliente sin reserva no recibe IP. Mira en el registro del servidor qué mensaje aparece.

Anexo A. El mismo servicio con Kea

Kea usa un fichero de configuración en **JSON**. Instalación en Debian 12:

```
sudo apt install kea-dhcp4-server
sudo systemctl status kea-dhcp4-server
```

Equivalente de nuestra configuración en /etc/kea/kea-dhcp4.conf:

```
{
  "Dhcp4": {
    "interfaces-config": {
      "interfaces": [ "ens19" ],
      "dhcp-socket-type": "raw"
    },
    "authoritative": true,
    "valid-lifetime": 600,
    "max-valid-lifetime": 7200,

    "lease-database": {
      "type": "memfile",
      "persist": true,
      "name": "/var/lib/kea/kea-leases4.csv"
    },

    "option-data": [
      { "name": "domain-name",          "data": "sri.local" },
      { "name": "domain-name-servers", "data": "1.1.1.1, 8.8.8.8" }
    ],

    "subnet4": [
      {
        "id": 1,
        "subnet": "192.168.29.0/24",
        "pools": [ { "pool": "192.168.29.100 - 192.168.29.199" } ],
        "option-data": [
          { "name": "routers", "data": "192.168.29.1" }
        ],
        "reservations": [
          { "hw-address": "bc:24:11:4c:7f:1d", "ip-address":
"192.168.29.50", "hostname": "pc-04" }
        ]
      }
    ]
  }
}
```

ISC DHCP (dhcpd.conf)	Kea (kea-dhcp4.conf)
INTERFACESv4 en /etc/default/...	interfaces-config dentro del propio fichero
default-lease-time / max-lease-time	valid-lifetime / max-valid-lifetime
subnet ... netmask ...	subnet4 con "subnet": "red/prefijo"
range	pools

ISC DHCP (dhcpd.conf)

Kea (kea-dhcp4.conf)

```
host { hardware ethernet ...; fixed-  
address ...; }
```

```
dhcpd -t -cf fichero
```

```
/var/lib/dhcp/dhcpd.leases
```

```
reservations
```

```
kea-dhcp4 -t fichero
```

```
/var/lib/kea/kea-leases4.csv
```

Validar y aplicar cambios:

```
sudo kea-dhcp4 -t /etc/kea/kea-dhcp4.conf
```

```
sudo systemctl restart kea-dhcp4-server
```

```
sudo journalctl -u kea-dhcp4-server -f
```

ACTIVIDAD 8: detén `isc-dhcp-server` (`sudo systemctl disable --now isc-dhcp-server`), instala Kea y configúralo con el fichero anterior. Comprueba que los clientes siguen recibiendo la misma configuración y compara el fichero de concesiones de Kea con el de ISC DHCP. ¿Qué formato te parece más fácil de procesar con un script?